1. Rod Cutting (考慮切的成本)

* p：直接賣價格 / r：該子問題的最高收益 / S：決定左邊該切哪

考慮此表：

i	1	2	3	4	5	6	7	8	9	10
P	1	3	6	6	10	9	8	13	8	12

原Rod Cutting 轉移式：

$$r(n)=\max_{1\le i\le n}\left(P_i+r(n-i)\right)$$

↓B.C.

i	0	1	2	3	4	5	6	7	8	9	10
P	0	1	3	6	6	10	9	8	13	8	12
r	0	1	3	6	7	10	12	13	16	18	20
s	0	1	2	3	1	5	3	1	3	3	5

$$r(1)=\max(\underbrace{P_1+r_0}_{1+0=1})=1$$

$$r(2)=\max(\underbrace{P_1+r_1}_{1+1=2},\underbrace{P_2+r_0}_{3+0=3})=3$$

⋮

$$r(4)=\max(\underbrace{P_1+r_3}_{1+6=7},\underbrace{P_2+r_2}_{3+3=6},\underbrace{P_3+r_1}_{6+1=7},\underbrace{P_4+r_0}_{6+0=6})=7$$

⋮

$r(10)=20$

Time : $O(n^2)$

```
1  CutRod(p, n):
2      create 2 array r、s
3      r[0] ← 0
4      for j ← 1~n:
5          now ← -∞
6          for i ← 1~j
7              if now < p[i]+r[j-i]:
8                  now = p[i]+r[j-i]
9                  s[j] = i
10         r[j] = now
11     return r, s
```

考慮「切」成本的 Rod Cutting：

$$r(n)=\max\left(P_n,\max_{1\le i<n}(P_i+r(n-i)-\underbrace{c}_{一刀2元})\right)$$

↓B.C.

i	0	1	2	3	4	5	6	7	8	9	10
P	0	1	3	6	6	10	9	8	13	8	12
r	0	1	3	6	6	10	10	11	11	14	18
s	0	1	2	3	4	5	3	2	3	3	5

$$r(1)=\max(\underbrace{P_1}_{1})=1$$

$$r(2)=\max(\underbrace{P_2}_{3},\underbrace{P_1+r_1-2}_{1+1-2})=3$$

⋮

$$r(6)=\max(\underbrace{P_6}_{9},\underbrace{P_1+r_5-2}_{1+10-2=9},\underbrace{P_2+r_4-2}_{3+6-2=7},\underbrace{P_3+r_3-2}_{6+6-2=10},\underbrace{P_4+r_2-2}_{6+3-2=7},\underbrace{P_5+r_1-2}_{10+1-2=9})=10$$

⋮

$r(10)=18$

L1 → CutRod(p,n,c)

L3 → now←P[j], S[j]←j

L7、L8 p[i]+r[j-i] ⟷ p[i]+r[j-i]-c

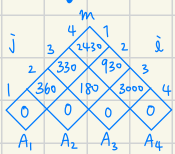
2. Matrix-Chain Multiplication

維度序列

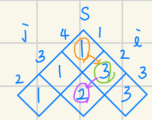
轉移式： $m(i,j)=\begin{cases}0, & i=j\\ \min\limits_{i\le k<j}\left(m(i,k)+m(k+1,j)+\underbrace{P_{i-1}P_kP_j}_{\text{維度序列}}\right) & i<j\end{cases}$

考慮此維度陣列： 10, 3, 12, 5, 50

matrix	A_1	A_2	A_3	A_4
dim	10x3	3x12	12x5	5x50



A_i~A_j最少乘積



最佳分割位置(最好的k)

→根據結果S進行查表

S(1,4) 切在 A_1 → 剩 A_2~A_4

S(2,4) 切在 A_3 → 剩 A_2~A_3

S(2,3) 切在 A_2 → 完成查表

→ $(A_1\times((A_2\times A_3)\times A_4))$ #

$m(1,2)=10\times3\times12=360$, $s(1,2)=1$

$m(2,3)=3\times12\times5=180$, $s(2,3)=2$

$m(3,4)=12\times5=50=300$, $s(3,4)=3$

$m(1,3)=\min(\underbrace{m(1,1)+m(2,3)+P_0P_1P_3}_{0+180+10\times3\times5=330},\ \underbrace{m(1,2)+m(3,3)+P_0P_2P_3}_{360+0+10\times12\times5=960})=330,\ s(1,3)=1$

⋮

$m(1,4)=2430$, $s(1,4)=1$

Time : $O(n^3)$

```
MatrixChain(p):
    n ← p.length()-1
    for i ← 1~n:
        m[i][i] ← 0
    for l ← 2~n:
        for i ← 1 ~ n-l+1:
            j ← i+l-1
            m[i][j] ← ∞
            for k ← i~j-1:
                q ← m[i][k]+m[k+1][j]+
                    P_{i-1}P_kP_j
                if q < m[i][j]:
                    m[i][j] ← q
                    s[i][j] ← k
    return m, s;
```

3. Longest Common Subsequence

轉移式： $c[i][j]=\begin{cases}0 & i=0 \text{ or } j=0\\ c[i-1][j-1]+1 & x_i=y_j \quad \text{Case 1}\\ \max(c[i-1][j],\ c[i][j-1]) & x_i\neq y_j \quad \text{Case 2、3}\end{cases}$

X= ABC

Y= BDC

	j	0	1	2	3
i		Y_j	B	D	C
0	X_i	0	0	0	0
1	A	0	↑0	↑0	↑0
2	B	0	↖1	←1	←1
3	C	0	↑1	↑1	↖2

=> Answer : BC

Step 1、 i=0 and j=0 全填0

$\underset{x_i\neq y_j}{C(1,1)}=\max(C[0][1],\ C[1][0])=0$

⋮

$\underset{x_i=y_j}{C(2,1)}=C[1][0]+1=1$

$\underset{x_i\neq y_j}{C(2,2)}=\max(\underset{0}{C[1][2]},\ \underset{1}{C[2][1]})=1$

⋮

$\underset{x_i=y_j}{C(3,3)}=C[2][2]+1$

Time : $O(mn)$

```
LCS(X, Y):
    m ← X.length(), n ← Y.length()
    for i ← 1~m:                  } i=0、j=0
        c[i][0] ← 0
    for j ← 0~n:
        c[0][j] ← 0
    for i ← 1~m:
        for j ← 1~n:
            if x_i = y_j:                  Case 1  x_i = y_j  左上長度+1
                c[i][j] ← c[i-1][j-1]+1
                b[i][j] ← "↖"
            else if c[i-1][j] ≥ c[i][j-1]:   Case 2、3  x_i ≠ y_j  上、左 取最大者
                c[i][j] ← c[i-1][j]
                b[i][j] ← "↑"
            else c[i][j] ← c[i][j-1]:
                b[i][j] ← "←"
    return b, c
```

4. Optimal Binary Search Trees

<決定哪個 key 當 root，和左右子樹怎麼長，能讓整棵樹期望搜尋成本最小>

轉移式： $e[i][j] = \begin{cases} q[i-1] & j = i-1 \\ \min\limits_{i \le r \le j} (e[i][r-1] + e[r+1][j] + w(i,j)) & i \le j \end{cases}$

子樹內機率和 $w[i][j] = w[i][j-1] + p[j] + q[j]$

*e：最小期望搜尋成本 / w：子樹內機率總和 (p·q)

$n=3$ / $P=[1,2,3,1]$ / $q=[1,1,1,1]$

w (i\j)	0	1	2	3
1	\	4	8	10
2	\	\	5	7
3	\	\	\	3
4	\	\	\	\

e (i\j)	0	1	2	3
1	1	6	15	21
2	\	1	7	13
3	\	\	1	5
4	\	\	\	1

⇒ Answer：$e[1][3] = 21$ $(e[1][n])$

Base Case

$l=1$

$e[1][1] = \min(e[1][0] + e[2][0] + w[1][1]) = 6$

$e[2][2] = \min(e[2][1] + e[3][2] + w[2][2]) = 7$

$l=2$

$e[1][2] = \min \begin{cases} r=1: e[1][0] + e[2][2] + w[1][2] = 16 \\ r=2: e[1][1] + e[3][2] + w[1][2] = 15 \end{cases} = 15$

⋮

$l=3$

$e[1][3] = 21$

⋮

Time: $O(n^3)$ / l 長度 / i 起点 / r 樹根

```
OptiBST(p, q, n):
    create 3 array e[1~n+1][0~n], w[1~n+1][0~n], root[1~n][1~n]
    for i ← 1~n+1:
        e[i][i-1] ← q[i-1]
        w[i][i-1] ← q[i-1]
    for l ← 1~n:
        for i ← 1~n-l+1:
            j ← i+l-1
            e[i][j] ← ∞
            w[i][j] ← w[i][j-1] + p[j] + q[j]
            for r ← i~j
                t = e[i][r-1] + e[r+1][j] + w[i][j]
                if t < e[i][j]:
                    e[i][j] ← t
                    root[i][j] ← r
    return e, root
```

5. Optimal Polygon Triangulation

<將一凸多邊形切成多個不重疊△，切割時，由三頂点產生的△会產生一權重 $w(v_i, v_j, v_k)$>

<利用多邊形內不相交的弦，將多邊形分割成 $n-2$ 个△，找出讓这 $n-2$ 个△權重和最小的切法>

轉移式：
$$t[i][j] = \begin{cases} 0 & i = j \\ \min\limits_{i \le k \le j-1} \left(t[i][k] + t[k+1][j] + w(v_{i-1}, v_k, v_j)\right) & i < j \end{cases}$$
=> 与 Matrix Chain 在數學上一致！

$n=4$ (4頂点), $w(v_0,v_1,v_2)=10$, $w(v_1,v_2,v_3)=12$,
$w(v_0,v_1,v_3)=11$, $w(v_1,v_2 v_3)=15$

t (i \ j)	1	2	3
1	0	10	23
2		0	12
3			0

→ $l=1$,

s (i \ j)	1	2	3
1	\	1	1
2	\	\	2
3	\	\	\

→ 先選 v_1 做切割点
Draw $\triangle v_0 v_1 v_3$
↳ 剩下左區間[1,1] & 右區間[2,3]
→ 再選 v_2 做切割点
Draw $\triangle v_1 v_2 v_3$
→ 完成！

$l=2$

$t[1][2] = \min(\underbrace{t[1][1]}_{0} + \underbrace{t[2][2]}_{0} + \underbrace{w(v_0,v_1,v_2)}_{10}) = 10 \rightarrow s[1][2] = 1$
$(k=1)$

⋮

$l=3$

$$t[1][3] = \min\begin{cases} k=1: t[1][1] + t[2][3] + w(v_0, v_1, v_3) = 23 \\ k=2: t[1][2] + t[3][3] + w(v_0, v_2, v_3) = 25 \end{cases} \rightarrow s[1][3] = 1$$

Time: $O(n^3)$

```
OptiTriang(v, w, n):
  create 2 array t[1~n][1~n], s[1~n][1~n]
  for i ← 1~n:
     t[i][i] = 0
  for l ← 2~n:
     for i ← 1~n-l+1:
        j ← i+l-1
        t[i][j] ← ∞
        for k ← i~j-1:
           q ← t[i][k]+t[k+1][j]+w(v_{i-1}, v_k, v_j)
           if q < t[i][j]:
              t[i][j] ← q
              s[i][j] ← k
  return t, s
```

6. 0-1 Knapsack Problem

轉移式：
$$dp[i][j] = \begin{cases} 0 & i=0 \,/\, j=0 \\ dp[i-1][j] & w_i > j \text{ 放不下} \\ \max(dp[i-1][j],\ dp[i-1][j-w[i]] + v[i]) & w_i \le j \text{ 放得下 拿or不拿} \end{cases}$$

$C=5$ / $(w_1, v_1) = (2,3)$、$(w_2, v_2) = (3,4)$、$(w_3, v_3) = (4,5)$、

dp (i \ j)	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7

Consider Obj_1:

$dp[1][1] = dp[0][1] = 0 \quad (2>1)$

$dp[1][2] = \max(dp[0][2], dp[0][0]+v_1) = 3$

$dp[1][3] = \max(dp[0][3], dp[0][1]+v_1) = 3$

⋮

Consider Obj_2:

$dp[2][1] = dp[1][1] = 0 \quad (3>1)$

$dp[2][2] = dp = [1][2] = 3 \ (3>2)$

$dp[2][3] = \max(\underbrace{dp[1][3]}_{3}, \underbrace{dp[1][0]+v_2}_{0+4}) = 4$

$dp[2][4] = \max(\underbrace{dp[1][4]}_{3}, \underbrace{dp[1][1]+v_2}_{0+4}) = 4$

⋮

Time: $O(nC)$

```
Knap01(v, w, n, C):
  create an array dp[0~n][0~C]
  for j ← 0~C:
     dp[0][j] = 0
  for i ← 1~n:
     dp[i][0] = 0
  for i ← 1~n:
     for j ← 1~C:
        if w[i] ≤ j:
           dp[i][j] ← max(v[i]+dp[i-1][j-w[i]],
                          dp[i-1][j])
        else
           dp[i][j] ← dp[i-1][j]
  return dp[n][C]
```